形式语言与自动机知识点整理

有穷状态自动机、正则表达式与正则语言性质

目录

1 有穷状态系统与有穷自动机 2

1.1 有穷状态系统 2

1.2 有穷自动机抽象装置 2

2 确定有穷自动机 DFA 2

2.1 DFA 的直观含义 2

2.2 DFA 的形式化定义 2

2.3 状态转移图与状态转移表 3

2.4 DFA 设计的典型问题 3

2.5 DFA 设计例题 3

2.6 扩展转移函数 5

2.7 DFA 接受的语言与正则语言 5

3 非确定有穷自动机 NFA 6

3.1 非确定转移 6

3.2 NFA 的形式化定义 6

3.3 NFA 的扩展转移函数 6

3.4 NFA 接受的语言 6

3.5 DFA 与 NFA 的对比 7

3.6 DFA 与 NFA 的等价性：子集构造法 7

4 带空转移的 NFA 7

4.1 epsilon-NFA 的含义 7

4.2 epsilon-NFA 的形式化定义 8

4.3 epsilon-闭包 8

4.4 epsilon-NFA 的扩展转移函数 8

4.5 epsilon-NFA 接受的语言 8

4.6 epsilon-NFA 与 DFA 的等价性 8

5 语言运算与正则表达式 9

5.1 语言运算 9
5.2 正则表达式的递归定义 9
5.3 运算符优先级 10
5.4 正则表达式设计例题 10
5.5 若干化简规则 11

6 有穷自动机与正则表达式的转换 11

6.1 等价性 11
6.2 DFA 到正则表达式：递归法 12
6.3 DFA 到正则表达式：状态消除法 12
6.4 正则表达式到自动机 13
6.5 正则表达式的代数定律 13

7 正则语言的泵引理 14

7.1 问题背景 14
7.2 鸽巢原理 14
7.3 泵引理内容 14
7.4 证明思路 14
7.5 用泵引理证明非正则的模板 15
7.6 典型示例 15
7.7 泵引理证明例题 15

8 正则语言的封闭性 16

8.1 封闭性的含义 16
8.2 并、连接、闭包 16
8.3 补运算 16
8.4 交运算 17
8.5 差运算 17
8.6 反转运算 17
8.7 同态 18
8.8 逆同态 18
8.9 封闭性用于证明非正则 18

9 正则语言的判定性质 18

9.1 典型判定问题 18
9.2 空性、有穷性和无穷性 19

9.3 正则语言等价性 19

10 DFA 最小化 **19**

10.1 状态等价性 19

10.2 填表算法 19

10.3 构造最小化 DFA 20

1 有穷状态系统与有穷自动机

1.1 有穷状态系统

有穷状态系统指：在系统运行过程中，系统状态在有限状态间不断变化；每个状态可以迁移到零个或多个状态；系统输入决定执行哪个状态迁移。

典型例子包括电视遥控器开关、交通信号灯、“狼、羊、菜问题”、数字电路设计、电脑游戏设计、通信协议（TCP、HTTP、Bluetooth、Wifi）、文本搜索、词法分析和编译器等。

1.2 有穷自动机抽象装置

有穷自动机作为抽象装置包含：

- 一条输入带：输入带划分为单元格，每个格子可以放置一个字符，整条输入带表示一个符号串；
- 一个读头：从左到右扫描输入带，将每次读到的字符传输给控制器；
- 一个有穷控制器：根据读入字符和当前状态进行状态转移，并控制读头读取下一个字符。

2 确定有穷自动机 DFA

2.1 DFA 的直观含义

确定有穷自动机（Deterministic Finite Automaton, DFA）具有有穷个状态。不同状态代表不同意义；输入字符串中的字符构成字母表；在任一状态下，从输入字符串中读入一个字符，根据当前状态和读入字符转到新的状态。DFA 有一个开始状态，也有一些终止状态或接受状态，表示到目前为止所读入字符构成的字符串是语言的一个句子。

2.2 DFA 的形式化定义

DFA 是五元组

$$A = (Q, \Sigma, \delta, q_0, F),$$

其中：

- Q：有穷状态集；
- Σ：有穷输入符号集或字母表；
- $\delta : Q \times \Sigma \to Q$：状态转移函数；

- $q_0 \in Q$：初始状态；
- $F \subseteq Q$：终结状态集或接受状态集。

2.3 状态转移图与状态转移表

DFA 常用两种直观方式表示。

- 状态转移图：每个状态对应一个节点；转移函数 $\delta(q_i, a) = q_j$ 表示为从 q_i 到 q_j 且标记为 a 的有向边；开始状态用标有 start 的箭头标识；接受状态用双圆圈表示。
- 状态转移表：每个状态对应一行，每个字符对应一列；若有 $\delta(q_i, a) = q_j$，则在第 q_i 行、第 a 列填入 q_j；开始状态前标箭头，接受状态前标星号。

2.4 DFA 设计的典型问题

典型问题是：设计 DFA，使其接受且仅接受给定语言，即某个符号串集合。
常见设计思路包括：

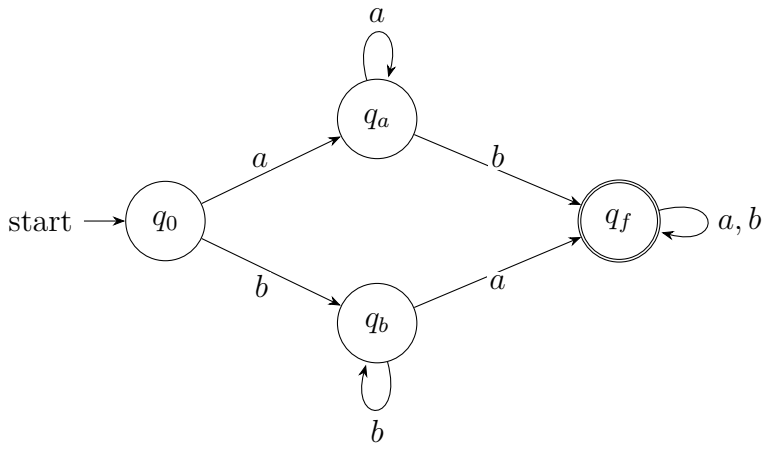
- 接受全部含有奇数个 1 的串：用状态记录当前已经读入的 1 的个数奇偶性；
- 接受全部含有偶数个 0 和偶数个 1 的串：用四个状态记录 (0 的奇偶性, 1 的奇偶性)；
- 接受含有 01 子串的全部串：状态记录“未发现 01 且 0 也未出现”“未发现 01 但刚读入 0”“已经发现 01”。

2.5 DFA 设计例题

例题 1：包含子串 ab 或 ba（2026 年期末考） 设 $\Sigma = \{a, b\}$，构造 DFA，接受所有包含子串 ab 或 ba 的字符串。

关键观察：在 $\{a, b\}$ 上，只要字符串中同时出现过 a 和 b，就必然出现相邻的 ab 或 ba。因此状态只需记录“尚未读入有效字符”“目前只见过 a”“目前只见过 b”“已经同时见过 a, b”。

$$Q = \{q_0, q_a, q_b, q_f\}, \quad F = \{q_f\}.$$



转移函数：

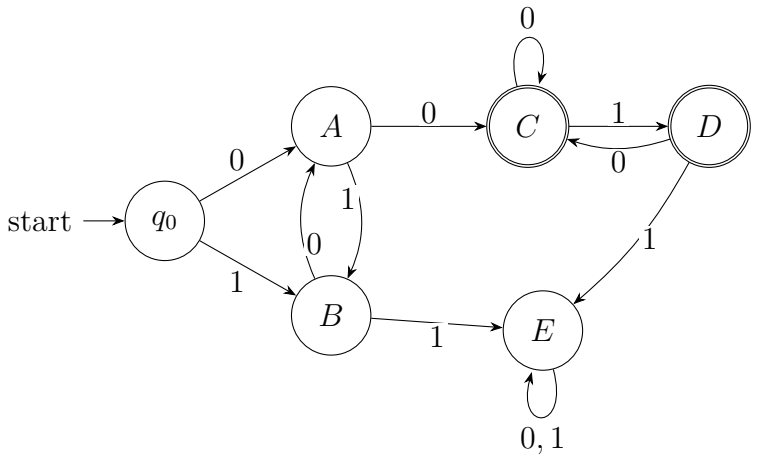
	a	b
q_0	q_a	q_b
q_a	q_a	q_f
q_b	q_f	q_b
q_f	q_f	q_f

例题 2：包含 00 且不包含 11（2026 年期末考） 设 $\Sigma = \{0, 1\}$，构造 DFA，接受所有包含连续两个 0 且不包含连续两个 1 的字符串。

状态含义：

$$Q = \{q_0, A, B, C, D, E\}, \quad F = \{C, D\}.$$

E 是死状态；C, D 表示已经见过 00，且当前还没有出现 11。



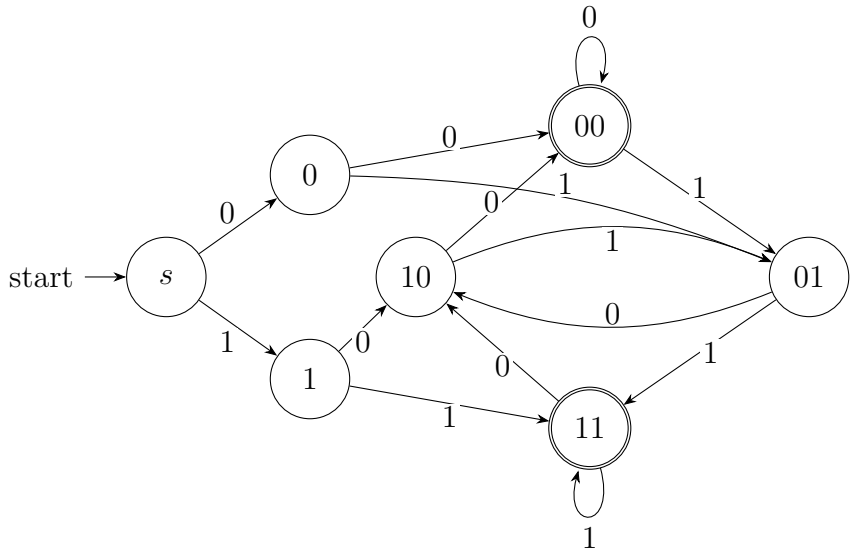
转移函数：

	0	1
q_0	A	B
A	C	B
B	A	E
C	C	D
D	C	E
E	E	E

例题 3：最后两个字符相同（2023 年期末考） 设 $\Sigma = \{0, 1\}$，构造 DFA，接受

$$L = \{w \mid w \in \{0,1\}^*,\ |w| \geq 2,\ w \text{ 中最后两个字符相同}\}.$$

构造思路：长度不足 2 时不能接受；长度至少 2 后，只需记录当前最后两个字符。接受状态为 00 和 11。



2.6 扩展转移函数

将状态转移函数扩展到字符串，定义扩展转移函数 $\hat{\delta}$：

$$\hat{\delta}(q, \varepsilon) = q,$$

$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a),$$

其中 a 是一个字符，w 是字符串。

2.7 DFA 接受的语言与正则语言

若 $A = (Q, \Sigma, \delta, q_0, F)$ 是一个 DFA，则 A 接受的语言为

$$L(A) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}.$$

即所有能使 DFA 从开始状态到达接受状态的符号串集合。

如果语言 L 是某个 DFA 的语言，则称 L 为正则语言。$\varnothing$、$\{\varepsilon\}$ 以及有限字符串集对应的语言都是正则语言。

3 非确定有穷自动机 NFA

3.1 非确定转移

在非确定有穷自动机中，同一状态在相同输入下可以有多个转移状态，自动机在某一时刻可以处在多个状态。因此给定输入和当前状态，状态转移函数的输出是一个状态集。

3.2 NFA 的形式化定义

NFA 是五元组

$$A = (Q, \Sigma, \delta, q_0, F),$$

其中：

- Q：有穷状态集；
- Σ：有穷输入符号集或字母表；
- $\delta : Q \times \Sigma \to \mathcal{P}(Q)$：状态转移函数；
- $q_0 \in Q$：初始状态；
- $F \subseteq Q$：终结状态集或接受状态集。

这里 $\mathcal{P}(Q)$ 是 Q 的幂集，即 Q 的所有子集构成的集合。

3.3 NFA 的扩展转移函数

将 δ 扩展到字符串，定义 $\hat{\delta}$。当输入串不为空时，先输入前面的字符串到达多个状态；对每个状态应用状态转移函数，然后把结果求并集：

$$\hat{\delta}(q, \varepsilon) = \{q\},$$

$$\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \delta(p, a).$$

3.4 NFA 接受的语言

若 $A = (Q, \Sigma, \delta, q_0, F)$ 是一个 NFA，则

$$L(A) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \varnothing\}.$$

也就是说，最后所处的状态集中有某个状态属于接受状态集时，该串被接受。

3.5 DFA 与 NFA 的对比

DFA	NFA
状态转移是确定的	状态转移是非确定的
一次状态转移只到一个确定状态	一次状态转移可能到多个状态
每个状态需要明确定义各输入符号的状态转移	不需要明确地定义所有状态转移；未定义转移表示 $\delta(q,a)=\varnothing$，该计算分支终止，只有显式补入死状态及相应转移后，才可说“进入 dead state”
有时设计较复杂	通常比 DFA 更容易设计
最后状态在接受状态集 F 中时接受	最后状态集中有某个状态在 F 中时接受
易于机器实现	表示语言能力与 DFA 等价

3.6 DFA 与 NFA 的等价性：子集构造法

语言 L 被 NFA 接受，当且仅当 L 被 DFA 接受。

给定 NFA

$$N=(Q_N,\Sigma,\delta_N,q_0,F_N),$$

构造 DFA

$$D=(Q_D,\Sigma,\delta_D,\{q_0\},F_D),$$

其中：

- $Q_D=\mathcal{P}(Q_N)$，即 NFA 状态集的每个子集都是 DFA 的一个状态；
- $\delta_D(S,a)=\bigcup_{q\in S}\delta_N(q,a)$；
- $F_D=\{S\subseteq Q_N\mid S\cap F_N\neq\varnothing\}$。

实际作图时，可删除不能从开始状态到达的无用状态；含有 NFA 接受状态的状态集就是 DFA 的接受状态。

4 带空转移的 NFA

4.1 epsilon-NFA 的含义

带空转移的 NFA（ε-NFA）是对 NFA 的扩展：在不输入任何字符的情况下，即输入空串 ε 时，可以自发地发生状态转移。它可以简化 NFA 的构造。状态表中会多出对应于 ε 的一列。

若不需要强调空转移，可将 ε-NFA 视为 NFA 的一种扩展形式。

4.2 epsilon-NFA 的形式化定义

ε-NFA 是五元组

$$A = (Q, \Sigma, \delta, q_0, F),$$

其中转移函数可写为

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q).$$

4.3 epsilon-闭包

状态 q 的 ε-闭包，记为 ECLOSE(q)，表示从 q 经过 ε 序列可到达的全部状态集合，即经过 0 个或大于 0 个空转移到达的状态集合。

递归定义：

1. $q \in \text{ECLOSE}(q)$，即包含当前状态本身；
2. 若 $p \in \text{ECLOSE}(q)$ 且 $r \in \delta(p, \varepsilon)$，则 $r \in \text{ECLOSE}(q)$。

状态集 S 的 ε-闭包定义为

$$\text{ECLOSE}(S) = \bigcup_{q \in S} \text{ECLOSE}(q).$$

4.4 epsilon-NFA 的扩展转移函数

对 ε-NFA，扩展转移函数要把空转移闭包纳入计算：

$$\hat{\delta}(q, \varepsilon) = \text{ECLOSE}(q),$$

$$\hat{\delta}(q, wa) = \text{ECLOSE}\left(\bigcup_{p \in \hat{\delta}(q,w)} \delta(p, a) \right).$$

4.5 epsilon-NFA 接受的语言

若 $A = (Q, \Sigma, \delta, q_0, F)$ 是一个 ε-NFA，则

$$L(A) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \varnothing \}.$$

4.6 epsilon-NFA 与 DFA 的等价性

语言 L 被 ε-NFA 接受，当且仅当 L 被 DFA 接受。

构造等价 DFA 时使用子集构造法并消除空转移。给定 ε-NFA

$$E = (Q_E, \Sigma, \delta_E, q_0, F_E),$$

可构造 DFA 的开始状态为 ECLOSE(q_0)，转移函数为

$$\delta_D(S,a) = \text{ECLOSE}\left(\bigcup_{q\in S}\delta_E(q,a)\right),$$

接受状态为

$$F_D = \{S \subseteq Q_E \mid S \cap F_E \neq \varnothing\}.$$

5 语言运算与正则表达式

5.1 语言运算

设 L 和 M 是两个语言，基本语言运算包括：

$$L \cup M = \{w \mid w \in L \text{ 或} w \in M\},$$

$$LM = \{xy \mid x \in L,\ y \in M\},$$

$$L^0 = \{\varepsilon\}, \qquad L^{n+1} = L^n L,$$

$$L^* = \bigcup_{i\geq 0} L^i.$$

其中 L^* 称为 Kleene 闭包。

例题：集合连接与差（2024 年期末考） 给定

$$A = \{\varepsilon, 01, 1\}, \quad B = \{01, \varepsilon\}, \quad C = \{011, 101\}.$$

1. 连接：

$$AB = \{xy \mid x \in A,\ y \in B\} = \{01, \varepsilon, 0101, 1, 101\}.$$

2. 差：

$$A^2 = AA = \{\varepsilon, 01, 1, 0101, 011, 101, 11\},$$

因此

$$A^2 - C = \{\varepsilon, 01, 1, 0101, 11\}.$$

5.2 正则表达式的递归定义

正则表达式通过代数式表示或产生正则语言，便于正则语言的计算机处理。正则表达式表示的语言与正则语言等价。

基础：

1. $\varnothing$ 是正则表达式，表示空语言；

2. ε 是正则表达式，表示语言 $\{\varepsilon\}$；

3. 对任意符号 $a \in \Sigma$，a 是正则表达式，表示语言 $\{a\}$，其有一个长度为 1 的字符串。

归纳：

1. 如果 E 和 F 是正则表达式，则 $E + F$ 也是正则表达式，表示 $L(E) \cup L(F)$；

2. 如果 E 和 F 是正则表达式，则 EF 也是正则表达式，表示 $L(E)L(F)$；

3. 如果 E 是正则表达式，则 E^* 也是正则表达式，表示 $L(E)^*$；

4. 如果 E 是正则表达式，则 (E) 也是正则表达式，表示同一语言。

5.3 运算符优先级

正则表达式中运算优先级从高到低为：

$$\text{括号}() \quad > \quad \text{星}* \quad > \quad \text{连接} \quad > \quad \text{加}+.$$

连接符通常省略。

5.4 正则表达式设计例题

例题 1：表达式含义（课堂例题） 正则表达式

$$(a + b)^*(a + bb)$$

定义的语言是：由 a 和 b 组成，仅以 a 或 bb 结尾的字符串集合。

例题 2：奇数长度字符串（2026 年期末考） 设 $\Sigma = \{a, b\}$，写出所有长度为奇数的字符串构成的语言的正则表达式。

长度为奇数可看作“先取 1 个字符，再接任意多个长度为 2 的块”，因此：

$$(a + b)\big((a + b)(a + b)\big)^*.$$

例题 3：长度为 3 的倍数且以 01 结尾（2026 年期末考） 设 $\Sigma = \{0, 1\}$，写出所有长度为 3 的倍数且以 01 结尾的字符串构成的语言的正则表达式。

结尾固定为 01，长度已经占 2 位。为了使总长度为 3 的倍数，前面还需要 $3k+1$ 位，即任意多个长度为 3 的块后再加一个任意字符：

$$\big((0 + 1)(0 + 1)(0 + 1)\big)^*(0 + 1)01.$$

例题 4：恰好有两个 a（2023 年期末考） 设 $\Sigma = \{a, b, c\}$，写出

$$L = \{w \in \{a, b, c\}^* \mid w \text{ 恰好有两个} a\}$$

的正则表达式。

两个 a 的前、中、后只能出现 b 或 c：

$$(b + c)^* a (b + c)^* a (b + c)^*.$$

例题 5：a 和 b 的总数为偶数（2024 年期末考） 写出语言

$$L = \{a^n b^m \mid n \geq 0,\ m \geq 0,\ n + m \text{ 为偶数}\}$$

的正则表达式。

由于字符串先是若干个 a，再是若干个 b，只需分两种情况：a 的个数和 b 的个数都为偶数，或都为奇数：

$$(aa)^*(bb)^* + a(aa)^* b(bb)^*.$$

5.5 若干化简规则

常用化简规则包括：

$$(\varepsilon + E)^* = E^*, \qquad (\varepsilon + E)^+ = E^*,$$

$$E + \varnothing = E, \qquad E\varnothing = \varnothing E = \varnothing,$$

$$E\varepsilon = \varepsilon E = E.$$

其中 $\varnothing$ 是连接运算的零元，ε 是连接运算的单位元。

6 有穷自动机与正则表达式的转换

6.1 等价性

DFA、NFA、ε-NFA 和正则表达式在表示语言的能力上是等价的：

- 对于一个 DFA 所能识别的语言，都存在一个正则表达式表示它；
- 任何一个正则表达式表示的语言，都可以由一个 ε-NFA 识别。

6.2 DFA 到正则表达式：递归法

定理：若 L 是某 DFA 的语言，那么存在正则表达式 R 满足 $L = L(R)$。
递归法思路：

- 对 DFA 的状态进行编号，令 1 为开始状态；
- 设 $R_{ij}^{(k)}$ 表示从状态 i 到状态 j，且中间节点状态编号不超过 k 的全部路径对应的字符串集合，即 k-路径；
- 基础 $R_{ij}^{(0)}$ 表示从 i 直接到 j、不经过任何中间节点的字符串集合；特别地，当 $i = j$ 时应包含零长度路径 ε，即 $R_{ii}^{(0)} = \{a \in \Sigma : \delta(i, a) = i\} \cup \{\varepsilon\}$，否则递推不保证与原自动机等价；
- 递归关系按“路径是否经过状态 k”分解：

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} \left(R_{kk}^{(k-1)}\right)^* R_{kj}^{(k-1)}.$$

如果 1 是开始节点，与 DFA 等价的正则表达式就是从状态 1 到所有终结状态的路径集合：

$$\sum_{f \in F} R_{1f}^{(n)}.$$

6.3 DFA 到正则表达式：状态消除法

状态消除法的核心思想：

- DFA 和从开始节点到结束节点所有路径对应的正则表达式集合等价；
- 若能把 DFA 转化成只含有开始节点和终止节点的形式，只需把开始节点到终止节点所有边上的正则表达式取出；
- 利用空转移增加新的开始状态和结束状态；
- 从 DFA 中逐个删除状态；
- 删除状态会消除原有路径，因此要补上等价路径；
- 用标记了正则表达式的新路径替换被删路径，保持自动机等价。

一般情况：对被删除状态 q 的每个入边和出边组合，都补一条等价新路径。若从 p 到 q 的边标记为 R_1，q 自环为 R_2，从 q 到 r 的边为 R_3，则需要为 p 到 r 补上

$$R_1 R_2^* R_3.$$

例如，接受全部以 01 结尾的串的 DFA，经状态消除可得到正则表达式

$$(1 + 0)^* 01.$$

6.4 正则表达式到自动机

定理：每个正则表达式定义的语言，都可被有穷自动机识别。更具体地，任何正则表达式 R 都存在与其等价的 ε-NFA，并且该 ε-NFA 满足：

- 仅有一个接受状态；
- 没有进入开始状态的边；
- 没有离开接受状态的边。

构造采用归纳法。
归纳基础：

- 对 $\varnothing$ 构造不接受任何串的 ε-NFA；
- 对 ε 构造只通过空转移接受空串的 ε-NFA；
- 对符号 a 构造读入 a 后从开始状态到接受状态的 ε-NFA。

归纳递推：

- $R+S$：添加一个开始状态和一个接受状态；将 R 和 S 通过空转移以并的方式连接起来；
- RS：先把 R 的接受状态改为非接受状态（保留其出边），再通过空转移将 R 的（原）接受状态与 S 的开始状态连接起来；
- R^*：添加一个新开始状态和一个新接受状态；增加从新开始状态到原开始状态的空转移（进入原自动机）；把原来的接受状态改为非接受状态，并通过空转移跳回原开始状态；通过空转移把原接受状态连接到新接受状态（即“闭包的一次及以上”）；闭包的 0 次由新开始状态到新接受状态的直接空转移实现。

6.5 正则表达式的代数定律

常用代数定律包括：

- 并的交换律：$E+F=F+E$；
- 并的结合律：$(E+F)+G=E+(F+G)$；
- 连接的结合律：$(EF)G=E(FG)$；
- 连接不满足交换律；
- 并运算的单位元：$E+\varnothing=E$；
- 连接运算的单位元：$E\varepsilon=\varepsilon E=E$；

- 连接运算的零元：$E\varnothing = \varnothing E = \varnothing$；
- 左分配律：$E(F+G) = EF + EG$；
- 右分配律：$(E+F)G = EG + FG$；
- 并的幂等律：$L + L = L$；
- 闭包相关：$(L^*)^* = L^*$，$\varepsilon^* = \varepsilon$。

7 正则语言的泵引理

7.1 问题背景

基本问题是：如何判定一个语言是否是正则语言？如果要证明语言 L 不是正则语言，直接证明没有任何 DFA、NFA 或正则表达式能够接受 L 很困难，因为需要排除无数模型。常用方法是使用泵引理。

7.2 鸽巢原理

鸽巢原理：若 n 只鸽子飞入 m 个巢且 $n > m$，则至少有一个巢中有不少于两只鸽子。有限状态、无限字符串的情况下，会导致路径中出现重复状态。

7.3 泵引理内容

泵引理：如果语言 L 是正则的，那么存在正整数 N，对任意 $w \in L$，只要 $|w| \geq N$，就可以将 w 分为三部分

$$w = xyz$$

满足：

$$|xy| \leq N, \qquad |y| > 0, \qquad xy^i z \in L \quad (i \geq 0).$$

也就是说，中间的串 y 不论循环几次，所得的串仍属于该正则语言。

泵引理成立是正则语言判定的必要条件，但不是充分条件。它可以用来确定特定语言不是正则语言。

7.4 证明思路

若 L 正则，则存在有 N 个状态的 DFA 接受 L。取 $w = a_1 a_2 \cdots a_m$ 且 $|w| \geq N$。当输入 w 的前 N 个字符时，经过 $N+1$ 个状态。由于 DFA 只有 N 个状态，由鸽巢原理，必有两个到达状态相同。

设重复状态之间的路径对应子串为 y。由于这段路径形成循环，循环可以重复任意多次；因此 $xy^i z$ 仍会到达接受状态。

7.5 用泵引理证明非正则的模板

1. 假设 L 是正则的；
2. 根据泵引理，存在正整数 N；
3. 选取一个特定的 $w \in L$，并保证 $|w| \geq N$；
4. 对任意满足 $w = xyz$、$|xy| \leq N$、$|y| > 0$ 的分解，分析 y 的形态；
5. 选取某个 i，通常是 0 或 2，使 $xy^iz \notin L$；
6. 这与泵引理要求 $xy^iz \in L$ 矛盾，因此 L 不是正则的。

7.6 典型示例

典型示例包括：

- 用泵引理证明形如 $\{0^n1^n \mid n \geq 0\}$ 的语言不是正则语言；
- 证明 $L_{eq} = \{w \mid w \text{ 由数量相等的0 和1 构成}\}$ 不是正则语言。取 $w = 0^N1^N$，因 $|xy| \leq N$，y 只能由 0 构成，泵入后 0 与 1 数量不再相等；
- 有限语言如 $\varnothing$、$\{00, 11\}$、$\{0^n1^n \mid 0 \leq n \leq 100\}$ 必定是正则语言，满足泵引理。可取 N 为有限语言最长字符串长度加 1，则不存在 $w \in L$ 且 $|w| \geq N$。

7.7 泵引理证明例题

例题 1：0 的个数多于 1 的个数（2023/2026 年期末考） 设

$$L = \{0^n1^m \mid 0 \leq m < n\}.$$

假设 L 是正则语言，则存在泵引理常数 N。取

$$w = 0^{N+1}1^N.$$

显然 $w \in L$，且 $|w| \geq N$。对任意分解 $w = xyz$，若满足 $|xy| \leq N$ 且 $|y| > 0$，则 x 和 y 都完全落在前面的 0 区域中，所以

$$y = 0^k, \quad k \geq 1.$$

取 $i = 0$，得到

$$xy^0z = xz = 0^{N+1-k}1^N.$$

此时 0 的个数为 $N + 1 - k \leq N$，而 1 的个数为 N，不再满足 0 的个数严格大于 1 的个数，因此 $xz \notin L$。这与泵引理矛盾，所以 L 不是正则语言。

例题 2：两端数量相等（2024 年期末考） 设

$$L = \{a^n b^\ell c^n \mid n \geq 0, \ell \geq 0\}.$$

假设 L 是正则语言，则存在泵引理常数 N。取

$$w = a^N c^N,$$

也就是令 $\ell = 0$。显然 $w \in L$，且 $|w| \geq N$。

对任意分解 $w = xyz$，若满足 $|xy| \leq N$ 且 $|y| > 0$，则 y 只能由 a 构成，即 $y = a^k$，$k \geq 1$。取 $i = 0$，得到

$$xy^0z = xz = a^{N-k}c^N.$$

此时 a 的个数少于 c 的个数，不满足两端数量相同，因此 $xz \notin L$。与泵引理矛盾，所以 L 不是正则语言。

8 正则语言的封闭性

8.1 封闭性的含义

正则语言经某些运算后得到的新语言仍保持正则，称正则语言在这些运算下封闭。

常见封闭运算包括：并、连接、闭包、交、反转、同态、逆同态、补、差。

8.2 并、连接、闭包

定理：正则语言在并、连接和闭包运算下保持封闭。

证明思路：由正则表达式的定义得证。若 L 和 M 是正则语言，存在正则表达式 R 和 S 表示它们，则 $R + S$、RS、R^* 分别表示并、连接和闭包后的语言。

8.3 补运算

定理：如果 L 是 Σ^* 上的正则语言，那么 $\overline{L} = \Sigma^* - L$ 也是正则的。

证明思路：设接受 L 的 DFA 为

$$A = (Q, \Sigma, \delta, q_0, F).$$

构造

$$A' = (Q, \Sigma, \delta, q_0, Q - F),$$

即把所有终止状态改成非终止状态，非终止状态改成终止状态，则 A' 接受 $\overline{L}$。

8.4 交运算

定理：如果 L_1 和 L_2 是正则语言，那么 $L_1 \cap L_2$ 也是正则语言。
一种证明使用补运算和并运算：

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}.$$

另一种证明通过 DFA 构造。若

$$A_1 = (Q_1, \Sigma, \delta_1, q_1, F_1), \quad A_2 = (Q_2, \Sigma, \delta_2, q_2, F_2),$$

则构造积自动机

$$A = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2),$$

其中

$$\delta((p, r), a) = (\delta_1(p, a), \delta_2(r, a)).$$

例如，若

$$L_1 = \{\text{由}\{0,1\}\text{ 组成的至少含有一个0 的字符串}\},$$

$$L_2 = \{\text{由}\{0,1\}\text{ 组成的至少含有一个1 的字符串}\},$$

则 $L_1 \cap L_2$ 是同时至少含有一个 0 和至少含有一个 1 的字符串集合。

8.5 差运算

定理：如果 L_1 和 L_2 都是正则语言，那么 $L_1 - L_2$ 也是正则的。
证明思路：

$$L_1 - L_2 = L_1 \cap \overline{L_2}.$$

由补和交的封闭性可得差运算封闭。

8.6 反转运算

字符串 w 的反转记为 w^R。语言 L 的反转定义为

$$L^R = \{w^R \mid w \in L\}.$$

定理：如果 L 是正则语言，那么 L^R 也是正则的。
两种证明思路：

- 对描述 L 的正则表达式进行归纳；
- 由识别 L 的 DFA 构造识别 L^R 的 NFA：将原初始状态改为唯一接受状态；把边调转方向；新增初始状态，并用空转移连到原接受状态。

8.7 同态

若 Σ 和 Γ 是两个字母表，同态定义为函数

$$h : \Sigma \to \Gamma^*,$$

即将 Σ 上的一个符号映射为 Γ 上的符号串。

扩展到字符串：

$$h(a_1 a_2 \cdots a_n) = h(a_1)h(a_2)\cdots h(a_n).$$

扩展到语言：

$$h(L) = \{h(w) \mid w \in L\}.$$

定理：若 L 是 Σ 上的正则语言，h 是 Σ 上的同态，则 $h(L)$ 也是正则的。

8.8 逆同态

若 $h : \Sigma \to \Gamma^*$ 是同态，且 L 是 Γ 上的语言，那么使 $h(w) \in L$ 的 w 的集合称为语言 L 的逆同态原像，记为

$$h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L\}.$$

定理：如果 h 是字母表 Σ 到字母表 Γ 的同态，L 是 Γ 上的正则语言，那么 $h^{-1}(L)$ 也是正则语言。

证明思路：由 L 的 DFA 构造识别 $h^{-1}(L)$ 的 DFA。新 DFA 每读入一个字符 a，相当于原 DFA 读入字符串 $h(a)$；若原 DFA 接受 $h(w)$，则新 DFA 接受 w。

8.9 封闭性用于证明非正则

可用封闭性间接证明非正则。例如，如果已知某语言不是正则，而另一个语言与它通过正则运算可联系起来，则可用反证法：假设目标语言正则，则经封闭运算得到的已知非正则语言也应正则，矛盾。

需要注意：如果 L_1 和 L_2 都不是正则的，并不能推出 $L_1 \cap L_2$ 一定不是正则的。

9 正则语言的判定性质

9.1 典型判定问题

正则语言或一般语言的典型判定问题：

- 以某种形式化模型描述的语言是否为空；
- 是否无穷；
- 某个特定串 w 是否属于所描述语言；
- 以两种方式描述的语言是否相同，即语言等价性。

9.2 空性、有穷性和无穷性

定理：具有 n 个状态的有穷自动机接受的集合 S 满足：

- S 非空，当且仅当它接受某个长度小于 n 的串；
- S 无穷，当且仅当它接受某个长度为 n 到 $2n-1$ 之间的串。

因此，对于正则语言：

- 存在算法判断其是否为空：只需检查全部长度小于 n 的串；
- 存在算法判断其是否无穷：只需检查全部长度从 n 到 $2n-1$ 的串。

9.3 正则语言等价性

定理：存在算法判定两个有穷自动机是否等价，即接受语言是否相同。
证明思路：设 A 和 B 分别接受 $L(A)$ 和 $L(B)$。考虑

$$(L(A)-L(B))\cup(L(B)-L(A)).$$

该语言是正则的，可由某个有穷自动机接受；它接受某个串当且仅当两个自动机语言不同。由于存在算法判断正则语言是否为空，因此可判定两个有穷自动机是否等价。

10 DFA 最小化

10.1 状态等价性

DFA 中两个状态 p 和 q，若对任意字符串 w，从 p 开始读入 w 被接受，当且仅当从 q 开始读入 w 也被接受，则称 p 和 q 等价；否则称为可区分。

等价性只要求两个状态对所有后续输入表现一致，即同时在接受或不接受意义上保持一致，不要求两个状态是同一状态。

10.2 填表算法

填表算法递归寻找 DFA 中全部可区分状态对：

1. 如果 $p\in F$ 而 $q\notin F$，或反过来，则 p 和 q 可区分；
2. 若存在输入字符 a，使得 $\delta(p,a)$ 和 $\delta(q,a)$ 是可区分的，则 p 和 q 可区分。

未被标记为可区分的状态对即为等价状态对。

10.3 构造最小化 DFA

根据填表算法得到的状态等价性，将 DFA 的状态集划分为不同的块。利用块构造新的 DFA：

- 新 DFA 的状态是等价类块；
- 新 DFA 的开始状态为包含原初始状态的块；
- 新 DFA 的接受状态为包含原接受状态的块；
- 新 DFA 的转移函数为块之间的转移。

得到的新 DFA 是原 DFA 的最小化 DFA。注意：最小化前应先删除不可达状态（从初始状态出发无法到达的状态），再做等价状态划分；若先合并等价类，合并后仍需删除不可达等价类，否则可能得不到最小的 DFA。另外，不能使用同样的方法最小化 NFA。